Energy and AI 26 (2026) 100896

Contents lists available at ScienceDirect

Energy and AI

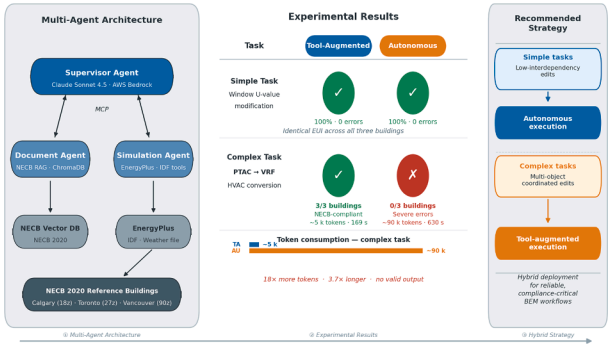
journal homepage: www.sciencedirect.com/journal/energy-and-ai

Tool-augmented vs. autonomous LLM agents for building energy model modifications: A comparative study using NECB compliance models

Navid Shirzadi *, Phylroy Lopez, Meli Stylianou

Natural Resources Canada, Ottawa, Ontario, Canada

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Large language models
Building energy modeling
EnergyPlus
Multi-agent systems
Model context protocol
Retrieval-augmented generation
NECB
AI-assisted design

ABSTRACT

This paper compares two large language model (LLM) agent architectures for modifying EnergyPlus Input Data Files (IDFs). In the tool-augmented approach, the LLM invokes high-level, domain-specific functions encapsulating heating, ventilation, and air conditioning (HVAC) engineering knowledge and IDF manipulation logic. In the autonomous approach, the LLM directly orchestrates primitive operations - object creation, deletion, and field-level modification. Both architectures are implemented as a multi-agent workflow using the Model Context Protocol (MCP), combining retrieval-augmented access to the National Energy Code of Canada for Buildings (NECB) standards, a simulation agent for IDF editing and EnergyPlus execution, and a coordinating supervisor agent. Seven NECB 2020 reference models spanning apartment, restaurant, school, warehouse, and office archetypes were evaluated on three modification tasks of increasing complexity. For a simple task (window U-value modification), both approaches achieved 100% success and produced identical energy results within each building. For a medium task (lighting-power reduction), both approaches succeeded in all seven buildings, although autonomous execution generally required more tool calls and time. For a complex task involving conversion from packaged terminal air conditioner (PTAC) or packaged direct-expansion (DX) systems to variable refrigerant flow (VRF), the tool-augmented approach succeeded in all seven buildings, producing simulation-ready, NECB-compliant models with zero severe errors. The autonomous approach failed in every case: the three apartment attempts stopped during planning or produced incomplete, invalid models, and the four packaged-DX attempts did not pass structural validation and therefore did not advance to annual simulation. Autonomous execution required substantially more tokens and agent processing effort, with no successful

* Correspondence to: Natural Resources Canada Ottawa, Ontario, Canada.
E-mail address: navid.shirzadi@nrcan-rncan.gc.ca (N. Shirzadi).

https://doi.org/10.1016/j.egyai.2026.100896

Available online 6 September 2026
2666-5468/© 2026 His Majesty the King in Right of Canada, as represented by Natural Resources Canada. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

complex-task outcome. These findings indicate that autonomous LLM agents are viable for low-interdependency edits, but reliability degrades sharply when tasks require many coordinated changes. A hybrid strategy is recommended: autonomous execution for simple modifications and validated domain-specific tools for complex, compliance-critical transformations.

1. Introduction

Building performance simulation (BPS) is central to energy-efficient building design because it allows architects, engineers, and policy makers to evaluate performance before construction. Most whole-building BPS tools share a common challenge: their input formats are highly structured, require considerable domain expertise to use correctly, and are prone to error even for experienced practitioners [1,2]. EnergyPlus, the de facto standard for detailed whole-building energy simulation, is no exception. Its Input Data Files (IDFs) encode building geometry, materials, heating, ventilation, and air conditioning (HVAC) systems, schedules, and controls through interdependent objects, making even routine changes time-consuming and error-prone.

Recent progress in large language models (LLMs) has created new possibilities for automating parts of this workflow. In their current state, LLMs now perform well at code generation, technical document interpretation, and multi-step reasoning [3], though this is a rapidly evolving field and their capabilities continue to advance. That capability raises a practical question for building energy modeling (BEM): should LLMs be allowed to manipulate building models directly, or should they be constrained to invoke validated, domain-specific tools?

That question remains underexplored in the literature. Most prior work on AI-assisted BEM emphasizes surrogate modeling, optimization, or energy prediction rather than the mechanics of editing simulation models themselves. Yet model modification is where many practical failures occur. This challenge is not unique to any single tool: across BPS platforms model objects are tightly interconnected, regulatory requirements such as the National Energy Code of Canada for Buildings (NECB) must still be satisfied after every change, and even small mistakes can make a model fail only at validation or simulation time. The NECB is Canada's national model code for energy efficiency in new buildings, covering prescriptive and performance-based requirements for envelope, lighting, and HVAC systems across eight climate zones; compliance with it is mandatory in most Canadian jurisdictions and serves as the benchmark against which the models in this study are evaluated [4,5]. This study focuses on EnergyPlus as a representative and widely adopted example, but the underlying challenge is a general feature of BPS practice. For that reason, the way an LLM is allowed to act is not just an implementation choice. It directly affects whether the resulting workflow is correct, repeatable, and practical to deploy.

1.1. Background and related work

1.1.1. Building energy modeling as a modification problem

BEM with EnergyPlus is a useful setting for studying LLM-based automation because the models are both expressive and structurally fragile. EnergyPlus represents buildings through highly structured IDFs whose objects encode envelope assemblies, schedules, HVAC equipment, controls, and simulation settings, often with many cross-references that must remain consistent after any edit [6]. In the Canadian context, those edits are further constrained by the NECB, which imposes climate-zone-dependent requirements for envelope performance, lighting, and HVAC efficiency under its prescriptive path and reference building approach [5]. As a result, direct model modification is not simply a text-editing task; it is a correctness-critical workflow in which local changes can have system-level consequences.

Most prior AI-assisted work in BEM has focused on adjacent problems rather than direct modification of simulation models. Reviews of the field show strong emphasis on prediction, calibration, optimization, and surrogate modeling, where machine learning is used to estimate building performance or accelerate design-space exploration [7–10]. That literature is important, but it does not resolve the practical problem addressed here: carrying out standards-aware, simulation-valid changes inside an existing EnergyPlus model. The distinction matters because real engineering workflows often depend less on generating new models from scratch than on revising established reference models while preserving regulatory compliance and simulation integrity. Earlier automation work points in the same direction. Kamel and Memari proposed the Automated Building Energy Modeling and Assessment Tool (ABEMAT) to streamline model generation and assessment from limited building information [11], while Ferrando et al. reviewed the broader ecosystem of urban building energy modeling (UBEM) tools and emphasized the workflow complexity involved in building, managing, and evaluating large sets of physics-based models [12]. These studies are relevant as precursors because they establish the importance of automation and workflow support in BEM, but they address a different problem from the one studied here: they do not examine LLM-based agents performing modification of an existing simulation model.

Beyond the AI-assisted literature, a substantial body of work has developed traditional, non-LLM automation approaches for BEM model generation and modification. These fall into three broad categories. First, rule-based and template-based generators - including ABEMAT [11] and integrated analysis platforms such as OpenStudio and its Measures ecosystem [13] - use pre-defined transformation scripts, decision trees, or ontology-driven templates to construct or modify EnergyPlus-compatible models from a small set of high-level design inputs. Second, parametric application-programming-interface (API) libraries exposed by tools such as the OpenStudio SDK [13] and by lower-level Python and Ruby IDF-manipulation packages [14] allow engineers to script arbitrary modifications while retaining direct control over object-level details, at the cost of requiring code-level familiarity with both the target platform and the applicable regulatory framework. Third, optimization-driven modification frameworks such as GenOpt [15] and its successors wrap the simulation engine in an outer search loop that iteratively edits parameter values to minimise an objective (energy use, cost, or comfort) through gradient-free or evolutionary methods, as surveyed in detail by Nguyen et al. [16]. Each of these traditional approaches offers deterministic and reproducible behaviour and, when the underlying scripts are code-compliant by construction, strong correctness guarantees. Their principal limitation, however, is that the modification workflow itself must be specified in advance by an engineer who is fluent in the target platform's API and the applicable code; they do not readily accommodate free-form modification requests expressed in natural language or novel task types that were not anticipated by the script author.

1.1.2. LLMs, retrieval, and structured tool use

Recent progress in natural language processing and code generation makes LLM-assisted model editing a plausible research direction. Retrieval-augmented generation (RAG) improves factual grounding by coupling generation with external document retrieval, which is especially relevant when an agent must follow building-code provisions and technical software documentation rather than rely on parametric memory alone [17,18]. In parallel, published systematic literature reviews in software engineering show that LLMs are becoming increasingly effective for code completion and program transformation, while also documenting persistent concerns around correctness, evaluation rigor, and dependable use in real workflows [19,20]. EnergyPlus editing

has that same profile: a change may be easy to describe in natural language while still being difficult to execute correctly across many dependent objects.

This tension is especially important in simulation workflows. A building model modification is not evaluated solely by whether text was generated or an object was created; it is evaluated by whether the resulting model remains internally consistent, code-aware, and simulation-ready. In such settings, the quality of an LLM workflow depends not only on the model's reasoning ability, but also on how much domain knowledge, validation logic, and dependency management are externalized into tools. This makes BEM a useful test case for comparing unconstrained generation against more structured, tool-mediated execution.

1.1.3. Agentic orchestration and the remaining gap

Another relevant stream of work concerns structured orchestration rather than unconstrained generation. In complex tasks, role specialization, external memory, and explicit coordination can improve performance by narrowing the action space exposed to the model and making intermediate decisions more inspectable [21]. In practical terms, agent-based workflows make it possible to separate standards retrieval, model inspection, simulation execution, and reporting into distinguishable components rather than collapsing all responsibility into one prompt-driven model [22].

More recent LLM-agent research also helps clarify what kinds of orchestration choices matter. In complex technical workflows, explicit decomposition of retrieval, planning, tool invocation, and validation can narrow the model's action space while making intermediate decisions easier to inspect and debug.

The emergence of common interaction protocols further strengthens this direction. The Model Context Protocol (MCP), for example, provides a standardized way to expose tools and resources to language-model-based systems, reducing integration friction and making tool invocation more explicit and auditable [23]. For technical domains such as BEM, this matters because reproducibility and validation depend heavily on whether external tools can be invoked consistently and whether intermediate actions can be inspected after the fact.

Recent BEM-specific LLM studies further clarify both the promise and the limits of the area. Zhang et al. developed an LLM-based agent schema and library for automated building energy analysis and modeling, showing that agent coordination can support parts of the modeling workflow [24]. Jiang et al. proposed Eplus-LLM, an LLM platform for automated building energy modeling, demonstrating that natural-language-driven generation of EnergyPlus models is increasingly feasible [25]. ElSayed et al. likewise emphasized usability and automation in rapid building energy model generation for non-expert-facing workflows [26]. Zhang et al. also explored LLMs as building energy modeling assistants through case studies, further supporting the view that natural-language interaction can accelerate modeling workflows while still leaving unresolved questions around validation, reliability, and control over downstream edits [27]. Together, these studies show growing momentum for LLM-assisted model creation and workflow automation in BEM.

Even so, the key comparison addressed in this paper remains sparsely covered in the literature. Existing background literature explains why BEM is difficult, why LLMs are attractive, and why retrieval or orchestration may help, but it still places higher emphasis on model generation, workflow automation, or high-level agent frameworks than on controlled comparison of modification strategies inside an already existing simulation model. In particular, prior work does not clearly distinguish between workflows in which the LLM invokes validated, domain-specific functions and workflows in which the LLM must directly sequence large numbers of primitive object-level edits.

This study sits at the intersection of these strands. It combines EnergyPlus-based building simulation, NECB-grounded retrieval, and MCP-based multi-agent orchestration, but evaluates a narrower and more operational question than most prior work: whether complex building model modifications are better handled through validated domain-specific tools or through largely autonomous sequences of primitive edits. Framed this way, the contribution of the paper is not simply that it applies LLMs to BEM, but that it isolates workflow structure itself as the object of comparison.

1.2. Research questions

Against this backdrop, this paper addresses four research questions: (1) how tool-augmented and autonomous LLM approaches compare in consistency, correctness, and compliance when modifying building energy models; (2) whether task complexity changes the relative performance of these approaches; (3) what computational trade-offs arise in token use and execution time and (4) which classes of model modification are best suited to each approach. Each of these questions is answered directly by the experimental results reported in Section 3, and their combined implications are used in Section 4 to derive a practical, hybrid deployment recommendation.

The paper makes five contributions: an experimental framework for comparing constrained and unconstrained LLM action surfaces in a technical domain; empirical evidence from a pilot-scale multi-building study under matched inference settings; an MCP-based multi-agent architecture integrating standards retrieval, IDF manipulation, and simulation; practical guidance for BEM practitioners adopting LLM-assisted workflows; and a hybrid deployment strategy grounded in observed task-complexity effects.

In addressing the four research questions above, this paper makes the following primary contributions:

- An experimental framework for the controlled comparison of LLM action surfaces in a technical, correctness-critical domain, in which the action surface exposed to the model is the single manipulated variable within the MCP experiment while the foundation model, retrieval grounding, prompting strategy, and simulation pipeline are held fixed across both conditions.
- Empirical evidence from a pilot-scale multi-building study under matched inference settings, quantifying differences in reliability and computational effort between the two approaches as task complexity increases.
- A reusable MCP-based multi-agent architecture integrating NECB standards retrieval (document agent), IDF manipulation and EnergyPlus execution (simulation agent), and coordinating supervision (supervisor agent), with all inter-agent communication conducted through the language-neutral Model Context Protocol so that both the LLM backbone and the orchestration framework are interchangeable and the workflow is portable to other LLM-based methods.
- Practical guidance for BEM practitioners adopting LLM-assisted workflows, based on the observed relationships among task interdependency, action granularity, validation burden, and execution cost.
- An evidence-grounded hybrid deployment recommendation - low-interdependency edits handled autonomously, high-interdependency and compliance-critical transformations routed through validated domain-specific tools

Critically, the primary novelty of this paper is not that it applies LLMs to BEM - prior work already does this - but that it contrasts two representative points on the action-surface / domain-knowledge continuum: a fully autonomous condition equipped only with primitive object-level operations, and a tool-augmented condition equipped with high-level, domain-encapsulating functions. The foundation model, retrieval grounding, prompting strategy, and simulation pipeline are held fixed across both conditions. As discussed in Section 4, the tool-augmented condition bundles coarser action abstraction with encapsulated

engineering and compliance knowledge, so the present study measures the joint effect of these two factors rather than isolating either one; the decomposition study needed to disentangle them is outlined as future work in the same section.

2. Methodology

This section is organised as follows. Section 2.1 walks through the end-to-end task workflow followed by every experimental run (Fig. 1). Section 2.2 describes the system architecture - the MCP communication layer, the three specialist agents (supervisor, document, simulation), and the underlying LLM inference configuration (Fig. 2). Section 2.3 defines the two action surfaces and their supporting algorithms. Section 2.4 introduces the seven NECB 2020 reference buildings. Section 2.5 specifies the light window, medium lighting, and complex HVAC tasks. Section 2.6 gives the evaluation protocol and feasibility rule, and Section 2.7 the computing platform and software stack.

2.1. Experimental workflow overview

The complete experimental workflow is shown in Fig. 1.

The flowchart depicts the following seven stages executed for each modification:

Stage 1: Initialization & MCP Session Setup: The workflow begins when a user submits a modification request. The supervisor agent establishes MCP connections to both the document agent (NECB RAG server) and simulation agent (EnergyPlus MCP server). Through the MCP's tool discovery protocol, the supervisor discovers available capabilities from each specialist agent. The experiment configuration file determines whether this particular run will use the tool-augmented or autonomous approach.

Stage 2: Standards Retrieval (NECB Compliance Query): The supervisor agent analyzes the modification request to identify applicable NECB compliance requirements. A structured query is sent via MCP to the document agent, which performs semantic search across the NECB vector database implemented in ChromaDB, an open-source embedding store and similarity-search system for retrieval-augmented applications [28]. The document agent returns relevant provisions including climate-zone-specific requirements, efficiency thresholds, and prescriptive rules. The supervisor aggregates these NECB constraints with the original modification request to form a compliance-aware context for the next stages.

Stage 3: Building Model Selection & Analysis: The simulation agent enumerates available NECB reference models through MCP. Based on the request parameters (building type, location), the appropriate model is selected. The agent loads the target IDF file and performs object discovery to understand the current building configuration. This analysis identifies existing HVAC systems, envelope components, and other objects that will be modified.

Stage 4: Modification Execution: At this stage, the workflow diverges based on the experimental approach:

Path A - Tool-Augmented Approach: The LLM examines the available tool library and selects the appropriate validated modification function (e.g., system conversion for HVAC modifications). The LLM provides high-level parameters such as efficiency targets and system type. The tool executes with built-in NECB compliance checks, automatic handling of object interdependencies, and validated operations. The tool returns a detailed modification report indicating success or failure with specific compliance verification.

Path B - Autonomous Approach: The LLM generates a sequence of primitive operations for direct IDF manipulation. The generated operations create, modify, or delete IDF objects based on the LLM's reasoning about requirements. The operations attempt to ensure reference integrity and syntax correctness. The simulation agent executes the operations and reports any syntax or reference errors. On failure, the LLM may regenerate operations with corrections (maximum 3 retry attempts). A

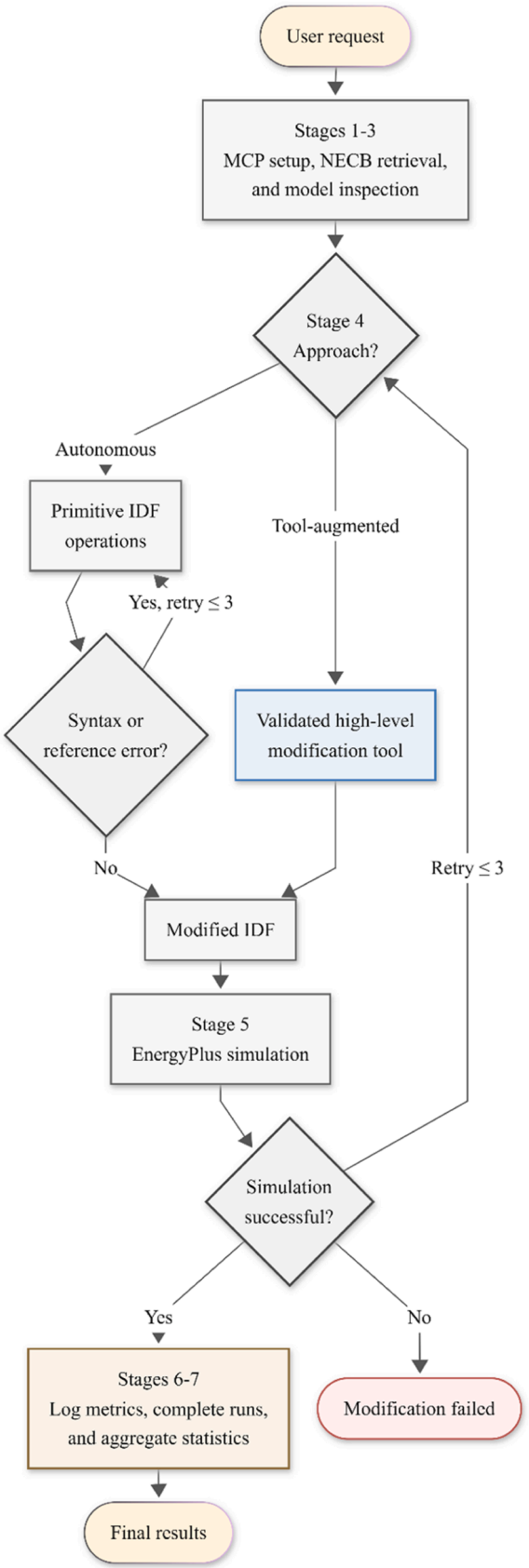


Fig. 1. Complete experimental workflow.

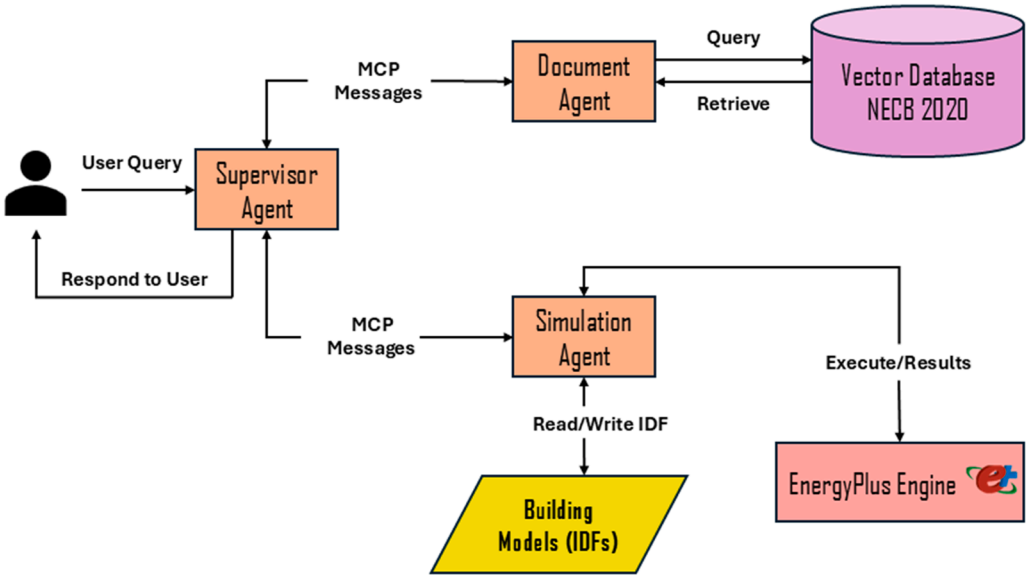


Fig. 2. Multi-agent architecture for LLM-based building energy modeling.

limit of three was chosen as it allows the model one initial attempt plus two correction passes, which was experimentally found to be sufficient to recover from isolated reference or naming errors, while preventing runaway token consumption on fundamentally ill-posed edits.

Stage 5: Simulation & Validation: Regardless of which path was taken, the modified IDF is saved to the output directory. The simulation agent invokes EnergyPlus through MCP, monitoring simulation progress and capturing any errors or warnings. If the simulation succeeds, the workflow proceeds to Stage 6. If it fails, the workflow returns to Stage 4 for error recovery (maximum 3 total attempts before marking the modification as failed). The same three-attempt cap applied here, consistent with the inner correction loop, so that total resource consumption remained bounded and outcomes across cases remained comparable.

Stage 6: Results Analysis & Logging: Successful simulations trigger energy metrics extraction. Because LLM outputs can vary stochastically across executions, the workflow records run-level metrics for later consistency checks and cross-case comparison. The supervisor compares these results against NECB requirements retrieved in Stage 2 to verify compliance, and logs the resulting experimental metrics, including IDF validity, simulation success status, token usage, operation count, and energy performance.

Stage 7: Cross-Case Comparison: After results are collected, outcomes are compared across buildings, task types, and modification strategies to identify performance patterns associated with modification complexity and workflow design.

2.2. System architecture

The experimental platform (Fig. 2) consists of three cooperating agents implemented using the MCP, all running Claude Sonnet 4.5 [29] via AWS Bedrock as the underlying language model. The supervisor agent receives user queries and coordinates workflow between two specialist MCP servers: the document agent providing NECB RAG capabilities with access to a vector database (ChromaDB), and the simulation agents managing building model repository and EnergyPlus execution. All inter-agent communication uses MCP standardized messages for tool discovery, invocation, and result aggregation.

2.2.1. MCP communication protocol

All agent interactions in the flowchart utilize MCP's standardized message format, with MCP error codes enabling structured retry logic as shown in the Stage 4 to 5 error recovery loop. This MCP-based architecture provides key properties essential for rigorous experimental comparison, including traceability via complete message logs capturing all agent decisions, scalability enabling parallel execution of independent building modifications, and extensibility allowing new capabilities to integrate without modifying the core workflow. When the LLM is configured with a temperature of zero - where temperature controls how tokens are sampled during text generation, with lower values producing more deterministic, repeatable outputs and higher values introducing more variation - the same architecture can also supports deterministic message sequences for fully reproducible runs; in the present study, all trials reported used temperature $= 0$ to isolate the architectural (tool-surface) variable from run-to-run sampling variance.

2.2.2. Document agent (NECB RAG)

The document agent serves as an MCP server providing standards-compliant guidance during building model modifications. It leverages NECB 2020 documentation as a data source, which is segmented into overlapping text chunks of 512 tokens with 64-token overlap and stored in a ChromaDB vector database using normalized 1024-dimensional embeddings generated by Amazon Titan Text Embeddings V2 through AWS Bedrock [30]. Amazon Titan Text Embeddings V2 is used consistently for both document ingestion and query encoding, ensuring document and query vectors remain in the same semantic space. The agent employs a top-5 retrieval strategy, returning the five most semantically relevant text chunks from the NECB vector database in response to each natural language query about climate zones, system types, and performance thresholds. Retrieved context is synthesized and interpreted to provide climate-zone-specific requirements and compliance guidance tailored to each modification task. This RAG implementation grounds agent responses in authoritative regulatory documents, ensuring modifications adhere to Canadian energy efficiency requirements.

2.2.3. Simulation agent

The simulation agent operates as an MCP server responsible for inspecting EnergyPlus IDF models, applying modifications, launching EnergyPlus simulations, and parsing the resulting outputs. Unlike the document and supervisor agents, it is primarily a tool-execution layer rather than a separate LLM-centered reasoning agent. High-level decisions that reach the simulation agent are still generated by the same LLM, while the simulation agent itself provides the execution

environment and tool interface for carrying out those decisions on EnergyPlus models. Within the experimental workflow, it provides the supervisor with a common interface for model selection, object discovery, parameter editing, simulation execution, and result extraction.

Two categories of tools are exposed by the simulation agent. Workflow-facing tools (Table 1) provide the shared task-execution interface used by both conditions. High-level HVAC tools (Table 2) are exposed only in the tool-augmented condition; they encapsulate the entire PTAC-to-VRF transformation, including NECB look-up, equipment sizing, performance-curve generation, object creation/deletion, reference management, and post-conversion validation. In the autonomous condition the high-level family is not exposed, and the agent is restricted to a small set of primitive IDF-operation tools (object search, inspection, creation, deletion, field-level modification, syntax validation, and working-copy saving) plus the workflow-facing tools of Table 1; the full inventory of these primitives and their signatures is listed in Algorithm 2 (Section 2.3), and their composition into a multi-step HVAC conversion is illustrated in Algorithm 3. The tool-augmented and autonomous conditions are therefore differentiated exclusively by whether the high-level HVAC family in Table 2 is exposed, in addition to the shared workflow tools.

Although Table 1 focuses on the shared workflow-facing tools, the experimental contrast between approaches is implemented through additional tool families. The autonomous condition relies on primitive operations such as object creation, deletion, inspection, and field-level modification, whereas the tool-augmented condition uses high-level HVAC-specific tools such as conversion preview and validated PTAC-to-VRF transformation. This separation is important because it shifts the burden of dependency management and compliance checking either to the LLM or to the tool itself.

2.2.4. Supervisor agent

The supervisor agent coordinates the multi-agent workflow and maintains conversation context throughout modification tasks. The supervisor routes queries to appropriate specialist agents based on task requirements, aggregates results from multiple agents into a coherent response, manages multi-turn interactions across the document retrieval and simulation execution phases, and checks task completion before returning results to the user. The supervisor operates as an MCP client, connecting to the specialist agent servers, discovering their available tools, invoking those tools as needed, and maintaining the overall task state while orchestrating the sequence of operations required to complete building model modifications.

Table 1
Workflow-facing simulation-agent tools (available in both conditions).

Tool Name	Purpose	Input Parameters	Output
list_building_models	Enumerate available NECB reference models	None	List of building model identifiers and metadata
read_idf_file	Load and parse IDF contents	Building model ID or file path	IDF object structure and field values
search_idf_objects	Discover modifiable objects in IDFs	IDF file, object type filter	Matching objects with names and properties
modify_idf_parameter	Execute IDF modifications	IDF file, modification specification	Modified IDF, validation status
run_energyplus_simulation	Execute simulation and collect results	IDF file, weather file, output variables	Simulation status, runtime, error/warning counts
parse_energyplus_results	Extract energy metrics from output files	Simulation output directory	Annual energy use, EUI, peak demands

Table 2
High-level HVAC conversion tool family (tool-augmented condition only).

Tool	Input	Encapsulated logic
get_necb_hvac_requirements	climate_zone, system_type, necb_version	NECB 2020 minimum-COP and control-requirement look-up (heating/cooling COP values reported in Section 2.5)
list_hvac_system_types	idf_file_path	Detection of existing HVAC archetypes (PTAC, VRF, split, etc.) and feasible conversion targets
preview_vrf_conversion	idf_file_path, climate_zone	Zone-load estimation, outdoor-unit sizing with 10% margin, heat-recovery heuristic, NECB compliance check
convert_ptac_to_vrf	source_idf, output_idf, climate_zone (and optional sizing overrides)	End-to-end transformation: remove PTAC + coils + fans, create VRF outdoor unit, performance curves, and per-zone terminal units, update equipment-list references, run reference-integrity check
convert_packaged_dx_to_vrf	source_idf, output_idf, climate_zone (and optional sizing overrides)	End-to-end packaged-DX transformation that preserves outdoor-air systems, air distribution units, baseboards, and valid setpoint-manager/node references

2.2.5. LLM configuration and inference scope

All buildings, tasks, and experimental conditions use the same pre-trained foundation model (Claude Sonnet 4.5 via AWS Bedrock) for inference-time reasoning and MCP tool selection. No model fine-tuning, gradient-based optimization, or weight updates are performed in this study. The experimental comparison is instead based on inference-time behavior under a fixed model backbone: operation sequencing quality, error propagation, retry outcomes, token usage, and execution time. The only intentional architectural difference between conditions is the action surface exposed to the same LLM: high-level HVAC tools (tool-augmented) versus primitive IDF operations (autonomous).

All reported trials used temperature = 0. Each individual trial was limited to a maximum of 12 LLM response turns, 45 tool calls, three tool errors, and a task-specific token ceiling. These settings were held constant across both conditions.

The choice of temperature = 0 is deliberate. The primary object of comparison in this study is the architectural variable - the action surface exposed to the model - rather than the run-to-run stability of any single condition. Holding the sampling policy fixed and deterministic across all runs ensures that any observed differences between the tool-augmented and autonomous conditions can be attributed to the tool surface rather than to seed-to-seed sampling variance. In a two-condition comparative pilot, introducing sampling variability alongside the architectural contrast would confound two independent sources of variance (architecture × sampling) and would require a substantially larger factorial design to interpret cleanly. Characterising seed-to-seed variance under non-zero-temperature sampling is therefore outside the scope of the present study; it is discussed as a scope restriction in Section 4 and is committed to as part of the follow-up experimental programme outlined there.

Each task was launched with a structured natural-language prompt specifying the target building, weather file, modification objective, and available tools. Critically, the same task description was used in both conditions; the only difference between the two prompts was the list of

available tools (high-level HVAC functions for the tool-augmented condition; primitive IDF operations for the autonomous condition).

The supervisor maintains a single linear conversation history per task, with tool inputs and outputs appended sequentially as MCP-formatted 'tool_use' and 'tool_result' blocks. No summarisation, truncation, or context compression is applied. The per-trial limits of 12 LLM response turns, 45 tool calls, three tool errors, and the applicable response-token ceiling bound agent execution.

On a tool call that raises an exception, the MCP server returns the error as a structured `is_error = true` tool result rather than raising an unhandled exception, so the model receives the error text in the next message and can attempt repair within its remaining retry budget.

2.3. Experimental approaches

This study compares two fundamentally different approaches to structuring LLM interaction with building energy modeling tool. These two approaches were evaluated on both modification categories used in the study. The simple task, window thermal transmittance modification, provides a low-interdependency baseline in which both approaches perform small numbers of localized edits. The complex task, PTAC-to-VRF conversion, is used in the algorithms below to illustrate how the same two approaches differ when many interdependent HVAC objects must be coordinated.

Approach 1: Tool-Augmented (Domain-Specific Function Library)

The LLM accesses a library of high-level, domain-specific functions that encapsulate building science and EnergyPlus expertise. These tools abstract away low-level IDF manipulation and embed knowledge about HVAC system types and their EnergyPlus object requirements, NECB compliance requirements by climate zone, object interdependencies and reference integrity, EnergyPlus autosizing for key capacity and airflow fields, rule-based parameter selection, and validated modification patterns. As shown in Algorithm 1, a single tool call such as ConvertHVACSystem() encapsulates the overall workflow: the function retrieves climate-zone-specific efficiency requirements, analyzes the source building structure, determines system sizing and operating parameters using a combination of EnergyPlus autosizing and rule-based estimates, iterates through each affected thermal zone to remove existing HVAC equipment and install new terminal units with appropriate connections, creates central system components such as outdoor units and supporting objects, generates performance curves meeting NECB minimums, and validates object references and compliance-related outputs before returning the modified IDF with compliance verification.

This tool-augmented approach exhibits several key characteristics that distinguish it from autonomous modification strategies. A single high-level function call can encapsulate hundreds of individual IDF object operations, with the exact number depending on building size and system complexity. In the present study, for example, the PTAC-to-VRF conversion tool encapsulated 208 object operations for Calgary Low-rise, 268 for Toronto Mid-rise, and 828 for Vancouver High-rise. The tools automatically handle object interdependencies and include built-in validation and error checking, while also relying on EnergyPlus autosizing for many key HVAC sizing fields. This requires the LLM to provide only high-level parameters such as climate zone and target system type. The LLM's decision space is thus limited to selecting which high-level tool to use and providing configuration parameters, while relying on the tool for all implementation details and domain-specific logic.

Algorithm 1: High-Level HVAC Conversion Tool (Tool-Augmented Approach)

Input: climate zone, source building file, target system type
Output: modified building file, validation report
Procedure: Convert HVAC System
Retrieve climate-zone-specific efficiency requirements
Load and analyze source building IDF structure
Determine system sizing and operating parameters using autosizing and rule-based estimates

(*continued on next column*)

(*continued*)

Algorithm 1: High-Level HVAC Conversion Tool (Tool-Augmented Approach)

for each thermal zone in building **do**
Remove existing HVAC equipment objects
Create new system terminal units with appropriate autosized fields and connections
Update zone equipment lists and connections
end for
Create central system components and required supporting objects
Generate performance curves meeting NECB minimums
Validate object references and compliance-related outputs
return modified IDF with compliance verification

Approach 2: Autonomous (Primitive Operation Library)

The autonomous condition exposes only low-level, general-purpose IDF manipulation operations with no embedded HVAC-specific transformation logic. In implementation terms, these operations correspond to reading model contents, searching for objects, creating new objects, deleting existing objects, inspecting object details, modifying individual fields, saving working copies, and invoking simulation and result-parsing tools for downstream validation. As shown conceptually in Algorithm 2, the LLM must decide which operations to invoke, in what order, and how to propagate resulting changes through the rest of the model. In this approach, the model is responsible for interpreting NECB requirements retrieved from the document agent, understanding EnergyPlus object relationships, sequencing the required edits, and managing object interdependencies without automated domain-specific assistance. Importantly, the autonomous agent is not a separately trained neural architecture; it is the same pre-trained LLM used in the tool-augmented condition, operating with a different tool interface and a larger orchestration burden.

For readability, Algorithm 2 uses generic operation names rather than the exact implementation-level tool names; in the actual system, these conceptual steps are realized through the simulation agent's primitive file, object, and validation tools.

Algorithm 2: Primitive Operations (Autonomous Approach)

AVAILABLE OPERATIONS:
ReadFile (filename) → file contents
SearchObjects (file, type) → objects
InspectObject (file, type, name) → object details
CreateObject (file, type, values) → object
ModifyField (file, type, name, field, value) → status
DeleteObject (file, type, name) → status
SaveWorkingCopy (source, output) → saved file
RunSimulation (file, weather) → status, errors, warnings
ParseResults (output) → metrics

Algorithm 3 illustrates the autonomous HVAC conversion strategy at a conceptual level. The workflow begins with an understanding phase in which the LLM queries NECB requirements, identifies existing PTAC equipment, and determines which thermal zones require conversion. It then proceeds through removal of the old system, creation of VRF outdoor equipment and performance curves, creation of zone-level terminal components, and a final validation-and-debugging phase. In practice, the number of primitive operations scales with the number of affected zones and dependent objects, so the action burden increases sharply as model size and interdependency complexity grow.

This autonomous approach differs from the tool-augmented strategy because planning, sequencing, dependency tracking, and error recovery remain the responsibility of the LLM rather than the tool. The LLM must manually preserve node connections, equipment lists, curve references, naming consistency, and compliance-relevant parameter values across a long sequence of edits. Methodologically, this means the autonomous condition places substantially more orchestration responsibility on the model itself, especially as the number of affected objects and cross-

references increases.

Algorithm 3: Autonomous HVAC Conversion Strategy

Input: climate zone, source file
Procedure: AutonomousHVACConversion
Phase 1: Understanding (10–15 operations)
Find all existing PTAC or packaged-DX systems
Identify all thermal zones
Phase 2: System Removal (approx. 100 operations)
for each old system **do**
Delete unit or conditioning components, cooling coil, heating coil, fan
end for
Phase 3: Outdoor Unit Creation (approx. 15 operations)
Create VRF outdoor unit with sizing parameters
for each required performance curve **do**
Create curve with coefficients
end for
Phase 4: Terminal Units (approx. 135 operations)
for each zone **do**
Create VRF terminal unit
Create cooling coil, heating coil, fan
Update zone equipment list
end for
Phase 5: Validation (approx. 30–50 operations)
Validate syntax
if errors exist **then**
Debug and fix iteratively
end if
return modified file

2.4. Dataset: NECB 2020 reference models

The experimental evaluation uses seven NECB 2020-compliant apartment reference models drawn from a suite of Canadian reference buildings adapted from the U.S. Department of Energy (DOE) commercial reference building prototypes [31,32] to represent distinct building scales and climate conditions. As summarized in Table 3, the cohort comprises apartment buildings in Calgary (Zone 7A), Toronto (Zone 5), and Vancouver (Zone 4), together with FullServiceRestaurant, SecondarySchool, Warehouse, and SmallOffice models in Montreal (Zone 6). The models range from 3 to 90 thermal zones. The three apartment models use zone-level PTAC systems with separate ventilation, whereas the FullServiceRestaurant, SecondarySchool, Warehouse, and SmallOffice models use 2, 20, 3, and 5 packaged-DX air loops, respectively.

The selected buildings therefore provide variation in model scale, baseline HVAC topology, and climate-zone-dependent compliance context. Increasing zone and equipment counts generally increases the number of repeated edits and dependent references that must be coordinated, while the PTAC and packaged-DX baselines test transformations involving both zone-level equipment and air-loop-connected systems. Although occupancy type varies across the cohort, all cases share the NECB 2020 code basis and are evaluated using the same three task definitions and matched experimental protocol, providing a common basis for comparison.

The workflow described in Section 2.2 is standards-agnostic: the supervisor, simulation agent, and MCP-based tool layer operate on any valid EnergyPlus IDF and perform no NECB-specific reasoning. Only two components carry NECB-specific content: (i) the document-agent knowledge base is currently indexed on NECB 2020, and (ii) the high-level HVAC conversion tool of Table 2 uses NECB minimum-COP look-up values. Both components can be re-populated to target a different code basis - for example, ASHRAE 90.1 for U.S. compliance or EN 15232 for a European context - without any change to the agent architecture, MCP tool servers, or evaluation harness. The choice of NECB 2020 in the present study therefore reflects a compliance-context selection for the Canadian pilot rather than an architectural constraint on the workflow itself; the same experimental workflow can be applied to any building energy model expressed as an EnergyPlus IDF, once the corresponding standards corpus and code-specific look-up values are supplied.

Table 3
Expanded experimental coverage.

Location	Climate Zone	Building Type	Stories	Floor Area (m^2)	Thermal Zones
Calgary	7A	Low-rise	3	1763	18
Toronto	5	Mid-rise	4	3134	27
Vancouver	4	High-rise	10	7834	90
Montreal	6	Full-service restaurant	1	511	3
Montreal	6	Secondary school	2	19,592	46
Montreal	6	Warehouse	1	4835	3
Montreal	6	Small office	1	511	5

2.4.1. Data characteristics

Because no supervised learning is performed, there is no train/validation/test allocation and no learned mapping between inputs and outputs. Instead, the system consumes three categories of input data and produces one category of output data per experimental run, as summarised in Table 4. The relationship between input and output is deterministic for a fixed EnergyPlus version: a given IDF and weather file always produce the same simulation result, so the relevant data-quality question is whether the modified IDF is internally consistent and simulation-valid rather than whether input-output pairs are correlated in a statistical sense.

2.5. Test case design

Three modification categories with increasing coordination requirements are evaluated (Fig. 3). The light task standardizes every 'WindowMaterial:SimpleGlazingSystem' U-factor to 1.5 W/m^2·K. The medium task reduces the active design-power input of every 'Lights' object by 30% while preserving schedule references. The complex task converts zone conditioning from PTAC or packaged-DX equipment to VRF while preserving ventilation, air-distribution, supplemental-heating, node, and control references. All three tasks were evaluated under both approaches on all seven buildings.

Operationally, the low-complexity case differs from the HVAC task

Table 4
Data characteristics of the experimental system.

Data role	Source	Format /Dimensionality	Scale per case
Building model input	NECB 2020 reference IDF	EnergyPlus IDF text; structured objects with named fields	3–90 thermal zones; file size ~1.0–4.5 MB
Standards corpus	NECB 2020 PDF	UTF-8 text, 512-token chunks with 64-token overlap; 1024-dimensional embeddings	Full NECB 2020 document; top-5 chunks retrieved per query
Weather input	Canadian Weather for Energy Calculations (CWEC) .epw	Hourly time-series: dry-bulb temperature, humidity, solar radiation, wind speed	8760 h × ~30 variables per city
Modification request	Natural-language task prompt	Text	Fixed per task type (window or HVAC)
Output: modified IDF	Simulation agent	EnergyPlus IDF text	Same dimensionality as input IDF; HVAC object count changes by task
Output: simulation result	EnergyPlus 23.2.0	Tabular HTML/CSV	Annual site energy intensity (MJ/m^2·yr), end-use breakdown, severe error count

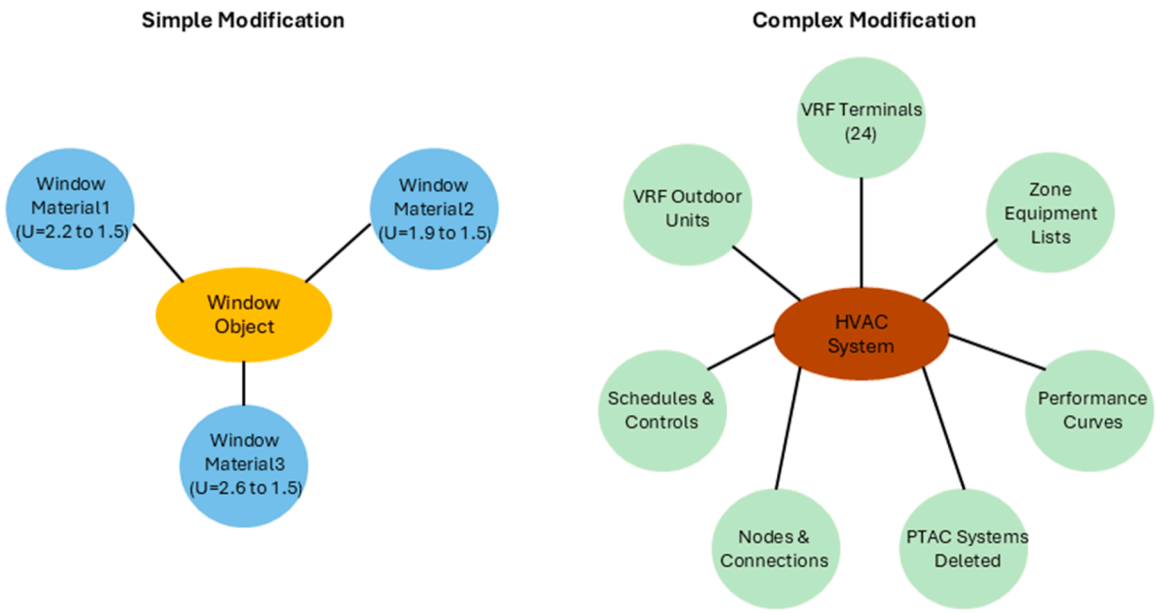


Fig. 3. Test case complexity hierarchy.

not only in the number of affected objects, but also in the burden placed on the LLM. In the tool-augmented condition, the window task can be completed through a single high-level parameter change, whereas in the autonomous condition it requires only a short sequence of primitive search and field-modification steps. This makes it an appropriate baseline for testing whether the two approaches remain comparable when coordination demands are minimal.

The complex category involves converting building HVAC systems from PTAC or packaged-DX systems to VRF heat pump systems. In the Toronto Mid-rise example, the 27 thermal zones comprise 24 dwelling-unit zones (8 units per floor across 3 floors) and 3 corridor zones. Each dwelling-unit zone is conditioned by one `ZoneHVAC:PackagedTerminalAirConditioner` object, which internally references a `Coil:Cooling:DX:SingleSpeed`, a `Coil:Heating:Water`, and a supply fan. Outdoor air for the 24 occupied zones is provided independently by a dedicated outdoor air system (DOAS, implemented as an `AirLoopHVAC` with an energy recovery ventilator) via constant air volume (CAV) terminal units without reheat; the 3 corridor zones are served by a separate mixed-air `AirLoopHVAC` with hot-water baseboard supplemental heat. Neither air-loop system is modified by the PTAC-to-VRF conversion. The conversion task targets only the 24 zone-level conditioning objects: the 24 `ZoneHVAC:PackagedTerminalAirConditioner` units and their associated coil and fan sub-objects are removed and replaced with 24 `ZoneHVAC:TerminalUnit:VariableRefrigerantFlow` units connected to a single new `AirConditioner:VariableRefrigerantFlow` outdoor unit, along with the required performance curves and updated `ZoneHVAC:EquipmentList` references. In the four Montreal archetypes, the packaged-DX-to-VRF path instead removes the baseline single-speed DX cooling components, creates one VRF terminal for each baseline air-distribution unit, and preserves the existing air-distribution units, outdoor-air systems, supplemental-heating equipment, nodes, and control references. Both paths present high interdependency complexity because equipment lists, node connections, performance curves, schedules, and retained air-system references must remain consistent throughout the conversion. NECB compliance is also central, since minimum VRF COP requirements vary by climate zone; in the implemented rule set, for example, the minimum heating COP is 5.9 in Zone 4, 6.3 in Zone 5, and 6.8 in Zone 7A. This category therefore serves as the study's high-complexity case, in which differences between high-level validated tools and primitive-operation orchestration can be observed under a much heavier coordination burden.

For reference, the engineering structure of the high-level HVAC system produced by either conversion path - the VRF heat-pump system itself, as opposed to the software tool that builds it - is summarised in

Table 5
Characteristics of the VRF HVAC system produced by the conversion.

Component	EnergyPlus Class	Key field	Expected value (s)
Outdoor condensing unit	AirConditioner: VariableRefrigerantFlow	Rated cooling capacity (W), rated heating capacity (W), rated cooling/ heating COP, defrost method, heat-recovery flag	Cooling COP 3.3–3.6; heating COP 5.9–6.8 (NECB minimum, zone-dependent); capacity autosized + 10% margin
Per-zone terminal unit	ZoneHVAC: TerminalUnit: VariableRefrigerantFlow	Supply-air flow rate (autosized), zone-level on/ off and modulation control	One unit per conditioned zone
Cooling coil	Coil:Cooling:DX: VariableRefrigerantFlow	Rated total capacity, rated SHR (autosized)	Autosized; SHR 0.70–0.80
Heating coil	Coil:Heating:DX: VariableRefrigerantFlow	Rated heating capacity (autosized)	Autosized to zone heating load
Supply fan	Fan:On/Off (or equivalent)	Rated airflow, total efficiency, pressure rise	Airflow autosized; efficiency 0.60–0.75; pressure rise 75–150 Pa
Performance curves	Curve:Biquadratic, Curve:Cubic, etc.	Capacity-modifying and EIR-modifying coefficients vs. temperature and PLR	Capacity ratio 0.5–1.2; EIR ratio 0.6–1.5; coefficients meet NECB minimum COP at rated conditions
Compliance constraint	(NECB rule set)	Minimum heating and cooling COP	Heating: 5.9/ 6.3/6.8 for Zones 4/5/7A; cooling: 3.3–3.6

Table 5. The system consists of a multi-zone outdoor condensing unit serving a set of per-zone terminal units linked by refrigerant piping, with performance characterised through capacity- and EIR-modifying curves and minimum COP thresholds set by NECB. Compared with the baseline PTAC and packaged-DX systems, this high-level HVAC configuration offers higher seasonal heating and cooling efficiency, per-zone modulation of refrigerant flow, and optional simultaneous heating and cooling between zones through a heat-recovery variant. These properties are the reason VRF is a representative "high-level" HVAC choice for the complex modification task in this study.

At the workflow level, this means the experimental contrast is not simply one call versus many calls. Rather, it compares a validated high-level abstraction that encapsulates dependency management against an autonomous sequence of object-level edits that must preserve references, naming consistency, control relationships, and compliance-relevant parameters across many linked HVAC components. The design therefore tests whether increasing coordination burden amplifies the advantage of higher-level tool abstraction.

2.6. Experimental protocol

Each of the three task categories was evaluated on all seven buildings under both approaches, yielding 42 building-task-approach cells. The reported dataset contains 50 agent trials in total, including repeated runs used to assess consistency. Validation for each trial included task-specific structural checking, EnergyPlus simulation using the appropriate weather file, NECB compliance review, and energy performance analysis. For packaged-DX conversion, structural validation required removal of all baseline single-speed DX cooling coils, creation of one VRF terminal per baseline air-distribution unit and at least one VRF outdoor unit, and preservation of all air-distribution units and outdoor-air systems. Structurally incomplete models were not simulated. The same LLM (Claude Sonnet 4.5), document agent, validation criteria, and per-trial execution limits described in Section 2.2.5 were applied across all conditions.

To support consistent cross-case comparison, the following performance metrics are reported: (1) task completion, expressed as the number of target objects successfully modified or converted out of the total required; (2) IDF validity, determined by whether the resulting IDF completed EnergyPlus simulation without severe errors; (3) severe error count from the final simulation attempt; (4) tool or operation count, the number of high-level tool calls in the tool-augmented condition or primitive operations in the autonomous condition; (5) agent processing time in seconds, defined as the workflow wall-clock time from the first LLM API call to the final workflow return, exclusive of EnergyPlus simulation runtime; (6) EnergyPlus simulation time in seconds, reported by EnergyPlus for the annual simulation; (7) total wall-clock time in seconds; (8) logical input-token consumption; (9) annual site energy intensity in MJ/m^2·year, reported only for trials producing a valid simulation; and (10) NECB compliance status. Metrics (5)-(7) distinguish LLM-side agent cost from deterministic EnergyPlus runtime so that the computational overhead attributable to autonomous reasoning can be examined separately from simulation cost.

Since the study is exploratory and case-comparative, the analysis emphasizes descriptive comparison rather than inferential statistics.

2.7. Computing platform and software stack

All experiments were run on a single workstation with the following configuration:

- Host operating system (OS): Windows 11 Pro x64
- Agent orchestration: Python 3.11; MCP inter-agent communication via the official `mcp` Python SDK over `stdio` transport [33]
- LLM backend: Anthropic Claude Sonnet 4.5 accessed via AWS Bedrock, region `us-east-1`
- IDF parsing: `eppy` 0.5.63 [34]
- Building energy simulation: EnergyPlus 23.2.0 (command-line executable invoked by the simulation agent) [35]
- Vector store: ChromaDB 0.5.x; embeddings generated by Amazon Titan Text Embeddings V2 (1024-dimensional) via AWS Bedrock
- Weather files: Canadian Weather for Energy Calculations (CWEC) `.epw` files for Calgary, Toronto, Vancouver, and Montreal

3. Results

Fifty agent trials were completed across three task categories, seven buildings, and two approaches. The results are organized by task category in Sections 3.2 (window U-value and lighting modifications) and 3.3 (PTAC- or packaged-DX-to-VRF conversion), and are then re-examined in Section 3.4 against the four research questions of Section 1.2. To make the alignment between the evidence and the research questions explicit, Table 6 identifies where the primary evidence for each research question is located.

3.1. Overview of experimental runs

Table 7 summarizes the six experimental condition types. Each condition was applied to all seven buildings, giving 42 building-task-approach cells and 50 agent trials analyzed in the remainder of this section. All reported trials used temperature = 0.

3.2. Window U-value modification results

All 14 window-modification trials - both approaches across all seven buildings - successfully updated every target WindowMaterial:SimpleGlazingSystem U-factor to 1.5 W/m^2·K, confirmed by direct inspection of the U-Factor field in each saved IDF, and completed EnergyPlus simulation without severe errors. All 22 lighting trials likewise reduced every active Lights design-power input by 30% while preserving schedule references and completed simulation with zero severe errors. Tables 8A and 8B report the full results.

For all seven buildings, both approaches modified every target window material and produced identical annual site EUI and site-energy change within each building, confirming that the action surface did not affect correctness for this localized edit. The tool-augmented approach required three calls in every building, whereas the autonomous approach required five calls in each apartment model and four in each Montreal archetype. Autonomous agent processing and total wall time were higher in every paired comparison, while simulation times remained similar within each building; the timing difference therefore arose primarily from orchestration rather than EnergyPlus execution. Across the full cohort, logical input ranged from 15,316 to 15,965 tokens for tool augmentation and from 10,201 to 13,750 for autonomous execution. The autonomous condition thus used more calls but fewer logical tokens in every window case, showing that logical-token consumption depended on response-turn sequencing and cached context rather than tool-call count alone.

Table 6
Mapping of research questions to primary evidence in results section.

RQ	Task Type	Primary evidence
RQ1	Consistency, correctness, compliance	Envelope and lighting outcomes in Tables 8A, 8B; HVAC outcomes in Tables 9A, 9B; RQ1 paragraph in Section 3.4
RQ2	Effect of task complexity	Light-to-medium contrast in Tables 8A, 8B and complex-task contrast in Tables 9A, 9B; RQ2 paragraph in Section 3.4
RQ3	Token and time trade-offs	Call, token, and timing fields in Tables 8A, 8B, 9A and 9B; RQ3 paragraph in Section 3.4
RQ4	Task–approach matching	RQ4 paragraph in Section 3.4 and hybrid deployment recommendation in Section 4

Table 7
Experimental task specifications.

Test ID	Task Type	Task Description	Target Specification	Approach	Tool Availability
W-TA	Window Modification	Modify window thermal transmittance to target value and validate through simulation	U = 1.5 W/m²·K	Tool-Augmented	High-level parametric modification functions, simulation tools, NECB document retrieval
W-AU	Window Modification	Modify window thermal transmittance to target value and validate through simulation	U = 1.5 W/m²·K	Autonomous	Primitive operations: object search, field modification, syntax validation, simulation execution
L-TA	Lighting reduction	Reduce every active Lights design-power input by 30% and preserve schedules	70% of baseline	Tool-Augmented	High-level lighting reduction and validation
L-AU	Lighting reduction	Apply the same reduction through object-level edits	70% of baseline	Autonomous	Primitive search, inspection, field modification, saving, and simulation
H-TA	HVAC Conversion	Convert building from PTAC to VRF heat pump system	NECB Zone 6-compliant VRF	Tool-Augmented	Domain-specific conversion, sizing, reference repair, and compliance checks
H-AU	HVAC Conversion	Convert building from PTAC to VRF heat pump system	NECB Zone 6-compliant VRF	Autonomous	Primitives: object creation, deletion, field modification, no embedded HVAC knowledge

Table 8A
Window U-value modification results. TA = Tool-Augmented; AU = Autonomous.

Building	Approach	Materials	Calls	Agent (s)	Simulation (s)	Tokens	EUI (MJ/m²·yr)
Calgary Low-rise	TA	3/3	1	22	76	15,965	552
Calgary Low-rise	AU	3/3	10	24	76	13,726	552
Toronto Mid-rise	TA	3/3	1	35	93	15,764	511
Toronto Mid-rise	AU	3/3	7	40	94	10,777	511
Vancouver High-rise	TA	3/3	1	118	331	15,825	429
Vancouver High-rise	AU	3/3	11	137	330	13,750	429
Montreal Full Service Restaurant	TA	2/2	3	28	23	15,360	1015
Montreal Full Service Restaurant	AU	2/2	4	29	22	10,205	1015
Montreal Secondary School	TA	2/2	3	115	289	15,319	549
Montreal SecondarySchool	AU	2/2	4	166	292	10,215	549
Montreal Warehouse	TA	2/2	3	30	22	15,347	483
Montreal Warehouse	AU	2/2	4	32	22	13,092	483
Montreal SmallOffice	TA	2/2	3	34	41	15,316	420
Montreal SmallOffice	AU	2/2	4	38	41	10,201	420

Table 8B
Lighting reduction: temperature-zero results. Completion counts refer to lights objects.

Building	Approach	Materials	Calls	Agent (s)	Simulation	Tokens	EUI (MJ/m²·yr)
Calgary Low-rise	TA	2/2	3	28	89	18,627	546
Calgary Low-rise	AU	2/2	7	82	83	34,473	546
Toronto Mid-rise	TA	2/2	3	34	104	18,628	518
Toronto Mid-rise	AU	2/2	4	126	110	10,653	518
Vancouver High-rise	TA	2/2	4	81	342	14,789	419
Vancouver High-rise	AU	2/2	4	321	353	10,642	419
Montreal Full Service Restaurant	TA	2/2	3	31	26	18,762	1011
Montreal Full Service Restaurant	AU	2/2	7	50	26	33,288	1011
Montreal Secondary School	TA	11/11	3	75	300	21,914	542
Montreal SecondarySchool	AU	11/11	16	598	301	60,549	543
Montreal Warehouse	TA	3/3	1	22	26	6840	496
Montreal Warehouse	AU	3/3	6	52	26	17,350	496
Montreal SmallOffice	TA	1/1	1	27	55	6628	422
Montreal SmallOffice	AU	1/1	3	39	51	9786	422

The lighting results extend the same pattern to a repeated-field task. Both approaches achieved complete structural and simulation success in all 22 temperature-zero trials, and every active lighting design-power input decreased by approximately 30%. After rounding to the reported precision, annual site EUI was identical between the two approaches in all seven buildings. Autonomous lighting required more calls in six of seven buildings, with equal call counts only in Vancouver, and its agent processing and total wall time were higher in every paired comparison. Because simulation times remained similar between approaches within each building, the additional elapsed time was attributable primarily to LLM orchestration rather than EnergyPlus execution. The largest coordination penalty occurred in the SecondarySchool model: autonomous execution required 16 calls, 598.2 s of agent processing, 60,549 logical input tokens, and 898.5 s total, compared with 3 calls, 75.8 s, 21,914 tokens, and 376.7 s for tool augmentation. This corresponds to approximately 7.9 times the agent-processing time and 2.4 times the total wall time. Logical-token use was nevertheless lower for autonomous execution in Toronto and Vancouver, reinforcing that token consumption was not proportional to call count. Thus, the medium task increased the cost of primitive orchestration without reducing completion reliability.

3.3. HVAC system conversion results

The HVAC conversion task revealed a complete separation between approaches: the tool-augmented method succeeded in all seven

buildings, while the autonomous method failed in all seven, although failure modes differed by baseline HVAC topology and building scale. Tables 9A and 9B report the full results for the three PTAC and four packaged-DX archetypes.

"–" indicates that no annual simulation was executed because the modified IDF failed validation (Tor-AU, Van-AU) or because no simulation was attempted (Cal-AU). The Cal-AU agent processing time (~30 s) is an estimate based on the run's < 2000-token consumption at the token-generation rate observed for the other autonomous HVAC runs; the agent terminated during the planning phase without invoking any modification tools, so no workflow wall-clock was separately captured in the archived log.

The tool-augmented approach completed the PTAC-to-VRF conversion fully in all three apartment buildings. Each resulting model passed validation, produced NECB-compliant VRF COP values (heating COP 6.8/6.8/6.5 and cooling COP 3.6/3.6/3.5 for Calgary/Toronto/Vancouver, respectively), and completed an annual EnergyPlus simulation with zero severe errors. Energy reductions ranged from 7.8% in Vancouver to 12.6% in Toronto. The number of IDF operations scaled with building size, from 208 in Calgary to 828 in Vancouver. Agent processing time was 169 s for Calgary and Toronto and 428 s for Vancouver, while token consumption remained at approximately 5000 tokens per run. The annual VRF simulation added 89–357 s of deterministic simulation time that would apply to any successful conversion regardless of the LLM condition.

The autonomous apartment approach produced qualitatively different outcomes at each scale. In Calgary, the agent recognized the scope of the required operations during planning and stopped before making substantive changes, so no simulation was attempted. In Toronto, the agent proceeded through 248 operations and converted 15 of 24 PTAC zones before reference and integration errors accumulated; the resulting IDF produced 27 severe errors. In Vancouver, the agent encountered object-naming and reference errors during large-scale creation, converted approximately one of 80 PTAC zones, and left the IDF unusable. Because no autonomous apartment conversion produced a valid IDF, none completed annual simulation.

The tool-augmented packaged-DX conversions also completed fully in all four Montreal archetypes. They created 2, 46, 3, and 5 VRF terminals for Restaurant, School, Warehouse, and Office, respectively, while preserving every baseline outdoor-air (OA) system and air distribution unit. All four models passed structural validation and annual simulation with zero severe errors, and their site-energy reductions ranged from 20.07% to 46.00%.

None of the four autonomous packaged-DX attempts passed the structural gate. Restaurant created both required terminals and an outdoor unit but left both baseline DX coils in place. Warehouse created only one of three required terminals and retained all three baseline DX coils. School reached its logical-input-token threshold without creating a VRF system, and Office stopped after three tool errors with no terminals created. No annual simulation was launched for these incomplete outputs. These are observed failures under the bounded protocol, not evidence that autonomous conversion is impossible with unrestricted resources.

The computational cost penalty of autonomous orchestration is visible in agent processing time and token use independently of simulation cost. In Toronto, autonomous execution required 630 s and approximately 90,000 tokens before failing, compared with 169 s and approximately 5000 tokens for tool augmentation - a 3.7-fold agent-time penalty and an 18-fold token penalty. In Vancouver, autonomous processing time exceeded the tool-augmented value by approximately 1.3-fold and token use by approximately 6-fold, while still producing no usable IDF. The four autonomous packaged-DX attempts consumed 160,986–366,310 logical input tokens and 88–516 s of agent processing, compared with 16,500–17,463 logical input tokens and 36–185 s for the successful tool-augmented runs. Applying published AWS Bedrock list prices for Claude Sonnet 4.5 at the time of study (approximately US$3 per million input tokens and US$15 per million output tokens) [36], the Toronto tool-augmented HVAC run cost on the order of US$0.02–0.08, while the Toronto autonomous run cost on the order of US$0.27–1.35 - an estimated 15–18-fold monetary penalty for a run that ultimately failed. The token counts in Tables 9A and 9B allow these estimates to be updated as list prices change.

3.4. Cross-case synthesis and answers to the research questions

The results across all three tasks and seven buildings reveal three consistent patterns. First, for the low-interdependency window task, the approaches were effectively equivalent in outcome across all buildings and climate zones: both succeeded, both produced valid simulations, and site energy intensities were identical within every building. Second, for the medium lighting task, both approaches again succeeded throughout the cohort, but autonomous execution generally required more calls and time as the number of repeated edits increased. Third, for the high-interdependency HVAC conversion, the approaches diverged completely: tool-augmented success was universal and autonomous failure was universal, with specific failure modes varying across PTAC and packaged-DX baselines. Building scale, occupancy, and climate zone did not independently determine the outcome: the same tool-augmented method succeeded in every building, and the same autonomous method succeeded on localized edits but failed on HVAC conversion. Using this synthesis, the four research questions of Section 1.2 can be answered directly from the experimental evidence.

RQ1: Consistency, correctness, and compliance. Within each condition, outcomes were consistent for the light and medium tasks across buildings and climate zones. Both approaches achieved 100% success on window modification and produced identical site energy intensities

Table 9A
PTAC-to-VRF HVAC conversion: apartment results. Cal = Calgary Low-rise; Tor = Toronto Mid-rise; Van = Vancouver High-rise; TA = Tool-Augmented; AU = Autonomous; EI = Site Energy Intensity.

Metric	Cal-TA	Cal-AU	Tor-TA	Tor-AU	Van-TA	Van-AU
PTAC zones converted	18/18	0/18	24/24	15/24	80/80	1/80
IDF operations	208	15	268	248	828	60
Simulation success	Pass	N/A	Pass	Fail	Pass	Fail
Severe errors	0	N/A	0	27	0	N/A (IDF unusable)
Agent processing time (s)	169	30	169	630	428	565
EnergyPlus simulation time (s)	89	–	107	–	357	–
Total wall-clock (s)	258	30	276	630	785	565
Token consumption	~5000	<2000	~5000	~90,000	~5000	~30,000
PTAC baseline EI (MJ/m^2 yr)	~554	N/A	526.95	N/A	~430	N/A
VRF EI (MJ/m^2 yr)	497.51	N/A	460.32	N/A	396.36	N/A
Energy reduction	10.2%	N/A	12.6%	N/A	7.8%	N/A
Heating / Cooling COP	6.8 / 3.6	N/A	6.8 / 3.6	N/A	6.5 / 3.5	N/A
NECB compliance	Compliant	Not achieved	Compliant	Not achieved	Compliant	Not achieved
Autonomous failure mode	–	Stopped at planning	–	4 error types; broken references	–	Object naming failures; abandoned

Table 9B
Packaged-DX-to-VRF HVAC conversion. Rest = FullServiceRestaurant; School = SecondarySchool; Whse = Warehouse; Office = SmallOffice; TA = Tool-Augmented; AU = Autonomous. Incomplete autonomous models did not advance to simulation.

Metric	Rest-TA	Rest-AU	School-TA	School-AU	Whse-TA	Whse-AU	Office-TA	Office-AU
VRF terminals / baseline units	2/2	2/2	46/46	0/46	3/3	1/3	5/5	0/5
OA systems retained	2/2	2/2	20/20	20/20	3/3	3/3	5/5	5/5
Baseline DX coils remaining	0	2	0	20	0	3	0	5
Simulation success	Pass	Not run	Pass	Not run	Pass	Not run	Pass	Not run
Severe errors	0	N/A	0	N/A	0	N/A	0	N/A
Tool calls	4	38	4	22	4	41	4	18
Agent processing time (s)	36	197	185	516	38	236	46	88
EnergyPlus simulation time (s)	34	–	646	–	21	–	49	–
Logical input tokens	16,500	219,152	17,463	366,310	16,543	259,302	16,645	160,986
VRF EUI (MJ/m^2·yr)	722.57	N/A	409.16	N/A	270.84	N/A	344.84	N/A
Energy reduction 29.8%		N/A	26.8%	N/A	45.9%	N/A	20.0%	N/A

within each building. Both also completed every lighting trial while preserving schedules and target values. For HVAC conversion, the tool-augmented approach succeeded in all seven buildings, producing structurally complete, NECB-compliant VRF systems with zero severe errors, whereas the autonomous approach failed to produce a simulation-valid conversion in any building. Consistency, correctness, and compliance therefore depended primarily on the combination of task type and action-surface condition rather than on a particular building instance.

RQ2: Effect of task complexity. Task complexity, measured by the number and interdependency of coordinated IDF operations, was the primary determinant of relative performance. For the low-interdependency window task, the two approaches were functionally interchangeable. Repeated lighting edits increased autonomous calls and wall time but did not reduce correctness. For HVAC conversion, the approaches diverged into a 100% versus 0% reliability split because the task required coordinated object creation, deletion, cross-referencing, control preservation, and standards application. The Secondary School comparison illustrates this transition: autonomous lighting completed all 11 field edits, whereas autonomous HVAC used 22 calls and 366,310 logical input tokens without creating a VRF terminal or outdoor unit.

RQ3: Computational trade-offs (tokens and time). For the window task, wall-clock times were comparable between conditions within each building, and EnergyPlus simulation dominated the total. For lighting, autonomous execution generally required more calls and was slower, reaching 16 calls, 60,549 logical input tokens, and 898.5 s in the Secondary School case. For HVAC conversion, autonomous orchestration consumed substantially more tokens and agent time yet produced no valid simulation. The apartment comparisons showed token penalties of approximately 6–18-fold, while the packaged-DX autonomous attempts used 160,986–366,310 logical input tokens compared with 16,500–17,463 for tool augmentation. The efficiency penalty of autonomous orchestration therefore compounded, rather than offset, the reliability penalty on the complex task.

RQ4: Task-approach matching. The evidence supports a task-dependent rather than uniform choice of approach. Low-interdependency, independently verifiable modifications such as window U-value changes can be handled by either condition at similar outcome. Repeated but structurally bounded lighting edits can also be handled autonomously, although with greater orchestration cost. High-interdependency, compliance-critical transformations involving equipment graphs, controls, node references, ventilation preservation, and standards look-ups were completed reliably only in the tool-augmented condition and should therefore be routed through validated domain-specific tools under comparable execution limits. This mapping is the empirical basis of the hybrid deployment recommendation in Section 4 and does not imply that primitive autonomous HVAC conversion is impossible with unrestricted resources.

4. Conclusion

This paper compared tool-augmented and autonomous LLM approaches for modifying building energy models using EnergyPlus IDFs. The comparison was carried out on seven NECB 2020 reference buildings: apartment models in Calgary, Toronto, and Vancouver, together with Full Service Restaurant, Secondary School, Warehouse, and Small Office models in Montreal. Three tasks with increasing structural complexity were evaluated: window U-value modification, lighting-power reduction, and conversion of PTAC or packaged-DX systems to VRF.

For the simple task of window U-value modification, both approaches performed well. They produced valid IDFs, completed EnergyPlus simulation successfully, and delivered matching energy results within every building. For the medium lighting task, both approaches again completed every target edit and simulation successfully, and annual site EUI was identical between approaches at the reported precision. Autonomous lighting generally required more calls and agent processing time, but this additional orchestration did not reduce correctness. Within the scope of this study, these results suggest that autonomous execution can be viable when the required edits are local, structurally bounded, and easy to validate.

For the complex task of PTAC- or packaged-DX-to-VRF conversion, the outcome was different. The tool-augmented approach completed all seven cases successfully and produced simulation-ready, NECB-compliant models with zero severe errors. The autonomous approach failed in every building, although the specific failure mode varied from planning-stage termination to partial conversion, broken references, retained baseline equipment, and exhaustion of the bounded tool-error or token budget. The difference was not due to lack of access to standards information. Rather, it arose from the difficulty of coordinating large numbers of dependent IDF operations without encapsulated domain logic. The results also indicate a practical efficiency penalty: for complex tasks, autonomous orchestration generally required much longer agent processing time and far greater token consumption, which may also translate into higher operating cost in real deployments. These are observed outcomes under the bounded protocol and should not be interpreted as proof that an unrestricted autonomous agent could never complete the task.

The main conclusion is therefore narrower but clearer: this study provides evidence which suggests that, currently, autonomous LLM agents can be effective for low-interdependency building model edits, but domain-specific tools remain the more reliable choice for complex, compliance-critical transformations. The most practical deployment strategy is therefore a hybrid one in which simple and structurally bounded tasks may be handled autonomously, while high-risk, multi-object modifications are routed through validated tools.

As a pilot study, the scope remains intentionally focused: seven buildings, three task types, one national code basis (NECB 2020), and one LLM family (Claude Sonnet 4.5). This focus allows a clean,

controlled comparison but means that broader generalisation requires further work. Extending the evaluation to additional building typologies, other HVAC conversion pairs, further climate zones and building codes, repeated trials per condition, and multiple LLM families will establish whether the patterns observed here hold across a wider operational range - and the experimental infrastructure developed in this study is directly reusable for that purpose. Instrumentation improvements for future runs include a dedicated pre-simulation IDF validation pass that produces an object-level invalid-object count, providing a more granular failure diagnostic than the EnergyPlus severe-error count alone.

A more fundamental methodological limitation concerns what the two experimental conditions actually vary, together with the sampling policy used to compare them. First, the tool-augmented condition differs from the autonomous condition along two dimensions simultaneously: (i) action abstraction, because a single high-level call replaces many primitive edits, and (ii) domain-knowledge encapsulation, because the high-level tool internally embeds NECB minimum-COP look-ups, sizing heuristics, performance-curve generation, and reference-integrity management. The present study therefore measures the joint effect of these two factors, and the reported HVAC reliability and efficiency gaps should be read as a comparison of two representative extremes on the action-surface/domain-knowledge continuum rather than as evidence that action-surface granularity alone determines outcome. Second, all 50 reported trials used temperature = 0 to reduce sampling variability and isolate the architectural comparison. Although the Restaurant lighting task was repeated five times per condition, broader run-to-run variability is not characterised. The observed HVAC effect size, 100% tool-augmented success versus 100% autonomous failure across seven buildings, together with approximately 6–21 times higher token use in the substantive failed autonomous comparisons, reflects repeated coordination breakdowns rather than a single near-miss error. Nevertheless, the stability of this result remains an empirical question that only broader repeated evaluation can settle.

A natural follow-up study is to introduce middle-tier tool families that decouple action abstraction from domain-knowledge encapsulation, for example, a node/reference connector (structural composition only, no engineering knowledge), a syntax and reference validator (correctness only, no compliance logic), and a code-compliance checker (regulatory logic only, no IDF construction). Comparing an autonomous agent equipped with these medium-grained tools against both the primitive and fully encapsulated conditions used here would isolate the marginal contribution of each factor. The same experimental matrix could support repeated trials at temperature = 0 and non-zero temperatures, as well as autonomous complex-task trials under explicitly larger budgets. Because the MCP-based architecture exposes each tool family as an independently discoverable server, new tool tiers can be added without changing the overall orchestration architecture. Together with the generalisation directions above, this decomposition represents the primary planned continuation of this work.

CRediT authorship contribution statement

Navid Shirzadi: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Phylroy Lopez:** Supervision, Conceptualization. **Meli Stylianou:** Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Crawley DB, Lawrie LK, Winkelmann FC, Buhl WF, Huang YJ, Pedersen CO, Strand RK, Liesen RJ, Fisher DE, Witte MJ, Glazer J. EnergyPlus: creating a new-generation building energy simulation program. Energy Build 2001;33:319–31. https://doi.org/10.1016/S0378-7788(00)00114-6.

[2] Beausoleil-Morrison I. Learning the fundamentals of building performance simulation through an experiential teaching approach. J Build Perform Simul 2019; 12:308–25. https://doi.org/10.1080/19401493.2018.1479773.

[3] Husein RA, Aburajouh H, Catal C. Large language models for code completion: a systematic literature review. Comput Stand Interfaces 2025;92:103917. https://doi.org/10.1016/j.csi.2024.103917.

[4] Huynh A, Dias Barkokebas R, Al-Hussein M, Cruz-Noguez C, Chen Y. Energy-efficiency requirements for residential building envelopes in cold-climate regions. Atmosphere 2021;12:405. https://doi.org/10.3390/atmos12030405.

[5] N.R.C. Canada, National Energy Code of Canada for Buildings 2020, (2022). https://nrc.canada.ca/en/certifications-evaluations-standards/codes-canada/codes-canada-publications/national-energy-code-canada-buildings-2020 (accessed April 16, 2026).

[6] Crawley DB, Lawrie LK, Winkelmann FC, Buhl WF, Huang YJ, Pedersen CO, Strand RK, Liesen RJ, Fisher DE, Witte MJ, Glazer J. EnergyPlus: creating a new-generation building energy simulation program. Energy Build 2001;33:319–31. https://doi.org/10.1016/S0378-7788(00)00114-6.

[7] Li X, Wen J. Review of building energy modeling for control and operation. Renew Sustain Energy Rev 2014;37:517–37. https://doi.org/10.1016/j.rser.2014.05.056.

[8] Westermann P, Evins R. Surrogate modelling for sustainable building design – a review. Energy Build 2019;198:170–86. https://doi.org/10.1016/j.enbuild.2019.05.057.

[9] Zhao H, Magoulès F. A review on the prediction of building energy consumption. Renew Sustain Energy Rev 2012;16:3586–92. https://doi.org/10.1016/j.rser.2012.02.049.

[10] Amasyali K, El-Gohary NM. A review of data-driven building energy consumption prediction studies. Renew Sustain Energy Rev 2018;81:1192–205. https://doi.org/10.1016/j.rser.2017.04.095.

[11] Kamel E, Memari AM. Automated building energy modeling and assessment tool (ABEMAT). Energy 2018;147:15–24. https://doi.org/10.1016/j.energy.2018.01.023.

[12] Ferrando M, Causone F, Hong T, Chen Y. Urban building energy modeling (UBEM) tools: a state-of-the-art review of bottom-up physics-based approaches. Sustain Cities Soc 2020;62:102408. https://doi.org/10.1016/j.scs.2020.102408.

[13] R. Guglielmetti, D. Macumber, N. Long, OPENSTUDIO: an open source integrated analysis PLATFORM, (n.d.).

[14] Gilani S, Haddad K, Kirney C, Lopez P, Syed A, Stylianou M. Performance of energy conservation measures in Canadian office buildings. Energy Convers Manag 2025; 330:119661. https://doi.org/10.1016/j.enconman.2025.119661.

[15] M. Wetter, Simulation research group building technologies department environmental energy technologies division Lawrence Berkeley National Laboratory Berkeley, CA 94720, (n.d.).

[16] Nguyen A-T, Reiter S, Rigo P. A review on simulation-based optimization methods applied to building performance analysis. Appl Energy 2014;113:1043–58. https://doi.org/10.1016/j.apenergy.2013.08.061.

[17] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: a survey, (2024). 10.48550/arXiv.2312.10997.

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-Augmented generation for knowledge-intensive nlp tasks, (n.d.).

[19] Martinez S, Xu L, Elnaggar M, Alomar EA. Software refactoring research with large language models: a systematic literature review. J Syst Softw 2026;235:112762. https://doi.org/10.1016/j.jss.2025.112762.

[20] Husein RA, Aburajouh H, Catal C. Large language models for code completion: a systematic literature review. Comput Stand Interfaces 2025;92:103917. https://doi.org/10.1016/j.csi.2024.103917.

[21] Park JS, O'Brien J, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative agents: interactive simulacra of human Behavior. In: Proceedings of the 36th annual ACM symposium on user interface software and technology. ACM; 2023. p. 1–22. https://doi.org/10.1145/3586183.3606763.

[22] Xu W, Wan H, Goel S, Antonopoulos CA. Development of a dynamic multi-agent network for building energy modeling: a case study towards scalable and autonomous energy modeling. Energy Build 2025:116712. https://doi.org/10.1016/j.enbuild.2025.116712.

[23] Introducing the model context protocol, (n.d.). https://www.anthropic.com/news/model-context-protocol (accessed April 14, 2026).

[24] Zhang L, Fu X, Li Y, Chen J. Large language model-based agent schema and library for automated building energy analysis and modeling. Autom Constr 2025;176: 106244. https://doi.org/10.1016/j.autcon.2025.106244.

[25] Jiang G, Ma Z, Zhang L, Chen J. EPlus-LLM: a large language model-based computing platform for automated building energy modeling. Appl Energy 2024; 367:123431. https://doi.org/10.1016/j.apenergy.2024.123431.

[26] ElSayed M, Shultz J, Kurtz J. User-friendly AI-driven automation for rapid building energy model generation. Energy Build 2025;345:116092. https://doi.org/10.1016/j.enbuild.2025.116092.
[27] Zhang L, Chen Z, Ford V. Advancing building energy modeling with large language models: exploration and case studies. Energy Build 2024;323:114788. https://doi.org/10.1016/j.enbuild.2024.114788.
[28] Introduction, Chroma Docs (n.d.). https://docs.trychroma.com/docs/overview/introduction (accessed April 15, 2026).
[29] Introducing claude sonnet 4.5, (n.d.). https://www.anthropic.com/news/claude-sonnet-4-5 (accessed April 21, 2026).
[30] Amazon titan text embeddings models - Amazon Bedrock, (n.d.). https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html (accessed April 16, 2026).
[31] M. Deru, K. Field, D. Studer, K. Benne, B. Griffith, P. Torcellini, B. Liu, M. Halverson, D. Winiarski, M. Rosenberg, M. Yazdanian, J. Huang, D. Crawley, U.S. department of energy commercial reference building models of the national building stock, 2011. 10.2172/1009264.
[32] C. Kirney, S. Gilani, K. Haddad, P. Lopez, M. Lubun, Initial models of existing Canadian buildings for building energy code analysis, in: 2023: pp. 819–27. 10.1007/978-981-19-9822-5_88.
[33] Modelcontextprotocol/python-sdk: the official python sdk for model context protocol servers and clients, (n.d.). https://github.com/modelcontextprotocol/python-sdk (accessed June 16, 2026).
[34] Running EnergyPlus from Eppy — Eppy 0.5.69 documentation. n.d. https://eppy.readthedocs.io/en/latest/runningeplus.html; 2026. accessed June 10
[35] EnergyPlus, (n.d.). https://energyplus.net/downloads (accessed June 17, 2026).
[36] Amazon bedrock pricing – AWS, Amazon Web Services, Inc. (n.d.). https://aws.amazon.com/bedrock/pricing/(accessed June 10, 2026).